Finals Buddy

Technical Documentation

An AI-powered adaptive finals-prep workspace

Architecture • Data Model • API • AI Agents

Developer / architecture reference

Contents

1 Overview **2**

2 Tech stack **3**

3 Repository layout **3**

4 Data model **4**
4.1 Schema migrations 5

5 Authentication **5**

6 AI agent layer **5**
6.1 Recommendation score 6

7 Ingestion pipeline **6**

8 RAG tutor **7**

9 BYOK + free-trial model **8**
9.1 Runtime key overrides 8

10 HTTP API reference **8**
10.1 Auth & account 8
10.2 Subjects 9
10.3 Materials & knowledge map 9
10.4 Study aids 9
10.5 Tasks, sessions, notes, tutor, dashboard 10
10.6 Admin (`ADMIN_EMAILS` only) 10

11 Frontend **10**

12 Observability **10**
12.1 Two rules the instrumentation follows 11
12.2 What it found immediately 11
12.3 Dashboard 11

13 Testing **12**

14 Configuration **12**

15 Running locally **12**

16 Deployment **13**
16.1 Infrastructure as code 13
16.2 Kubernetes 14
16.3 CI/CD 14
16.4 Migration history 15

1 Overview

Finals Buddy is an AI study companion. A user uploads course material (PDF, DOCX, PPTX, TXT); the backend parses it, indexes it for retrieval, and runs a chain of LLM agents to produce a summary, flashcards, quizzes, mock exams, formula sheets, and a knowledge map — all grounded in the user's own material. A RAG tutor lets the user chat with their material, and a recommendation engine ranks what to study next.

For the product pitch, feature list, and screenshots, see the root `README.md`. This document covers how the system is built.

Two-service architecture:

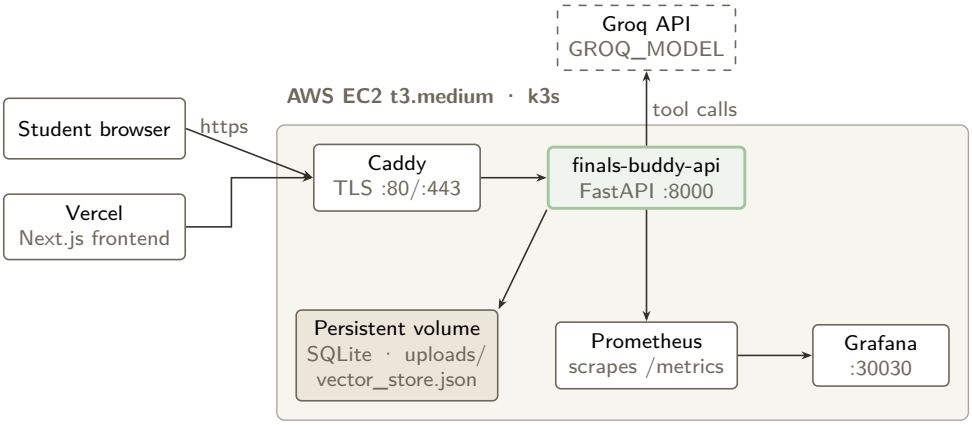


Figure 1: One EC2 node runs the API, its TLS front door, and its own monitoring. Only ports 80 and 443 are public; Grafana is restricted to the operator's IP.

The backend is deliberately self-contained: SQLite for relational data, a file-backed TF-IDF vector store for RAG (no external vector DB), and Groq as the only external AI dependency. Everything that must survive a redeploy lives on the one persistent volume.

The environment is described in Terraform (`infra/`), and deployments arrive through GitHub Actions over AWS Systems Manager — there is no SSH key and port 22 is closed.

2 Tech stack

Layer	Technology
Frontend	Next.js 16, React 19, TypeScript 5, Tailwind CSS 4
Rich editor	TipTap 3 (+ math extension, KaTeX)
Icons	lucide-react
Backend	FastAPI, Uvicorn, Python 3.12
ORM / DB	SQLAlchemy 2, SQLite
LLM	Groq, model set by `GROQ_MODEL` (default `qwen/qwen3.8-27b`) via groq SDK + LangChain
Agent loop	LangChain / langchain-groq
RAG retrieval	Custom local TF-IDF cosine store (`vector_store.py`)
Doc parsing	pypdf, docx2txt, python-pptx
Crypto	`cryptography` (Fernet) for BYOK key encryption
Metrics	prometheus-client (`app/observability.py`)
Orchestration	k3s (single-node Kubernetes)
TLS	Caddy, automatic Let's Encrypt
Monitoring	Prometheus + Grafana, in-cluster
IaC	Terraform (EC2, ECR, IAM, SSM Parameter Store)
CI/CD	GitHub Actions, OIDC to AWS, deploy over SSM
Deploy	Docker → ECR → k3s on AWS EC2; frontend on Vercel

Note on Next.js. `frontend/AGENTS.md` warns this is a newer Next.js with breaking changes vs. older conventions — consult `node_modules/next/dist/docs/` before writing frontend code.

3 Repository layout

```
finals-buddy/
|-- backend/
|   |-- app/
|   |   |-- main.py          # FastAPI app + all HTTP endpoints (~1700 lines)
|   |   |-- models.py        # SQLAlchemy ORM models (the data model)
|   |   |-- schemas.py       # Pydantic request/response schemas
|   |   |-- database.py      # engine, SessionLocal, get_db dependency
|   |   |-- config.py        # env config + runtime Groq-key overrides
|   |   |-- auth.py          # PBKDF2 hashing + HMAC bearer tokens
|   |   |-- key_context.py   # BYOK + free-trial gating (contextvar routing)
|   |   |-- admin.py         # /api/admin/* router
|   |   |-- errors.py        # AIServiceError
|   |   |-- observability.py # Prometheus metrics for the RAG/LLM layer
|   |   `-- services/
|   |       |-- agents.py         # LLM agents (summarize, quiz, exam, ...)
|   |       |-- langchain_chat.py # RAG tutor (Groq tool-calling loop)
|   |       `-- vector_store.py   # local TF-IDF vector store
|   |-- tests/
|   |   |-- test_rag_eval.py      # retrieval quality gate (golden set)
|   |   `-- test_observability.py # metrics recording + safety
|   |-- Dockerfile
|   `-- requirements.txt
|-- infra/
|   |-- terraform/           # EC2, ECR, IAM (OIDC), SSM parameters
```

```
|   |-- k8s/                  # app, Caddy, Prometheus, Grafana manifests
|   `-- scripts/              # apply-manifests, populate-secrets, seed-demo
|-- frontend/
|   `-- src/
|       |-- app/
|       |   |-- page.tsx              # home / subject list
|       |   |-- login/page.tsx
|       |   |-- admin/page.tsx
|       |   `-- subject/[id]/page.tsx # main study workspace (~3800 lines)
|       |-- components/               # AccountSettings, NotionEditor, Toast
|       `-- lib/api.ts                # typed API client + session handling
|-- docs/                             # screenshots + this document
`-- README.md
```

4 Data model

All tables are defined in `backend/app/models.py`. `User` owns `Subject`s; almost everything else hangs off `Subject` with `ON DELETE CASCADE`.

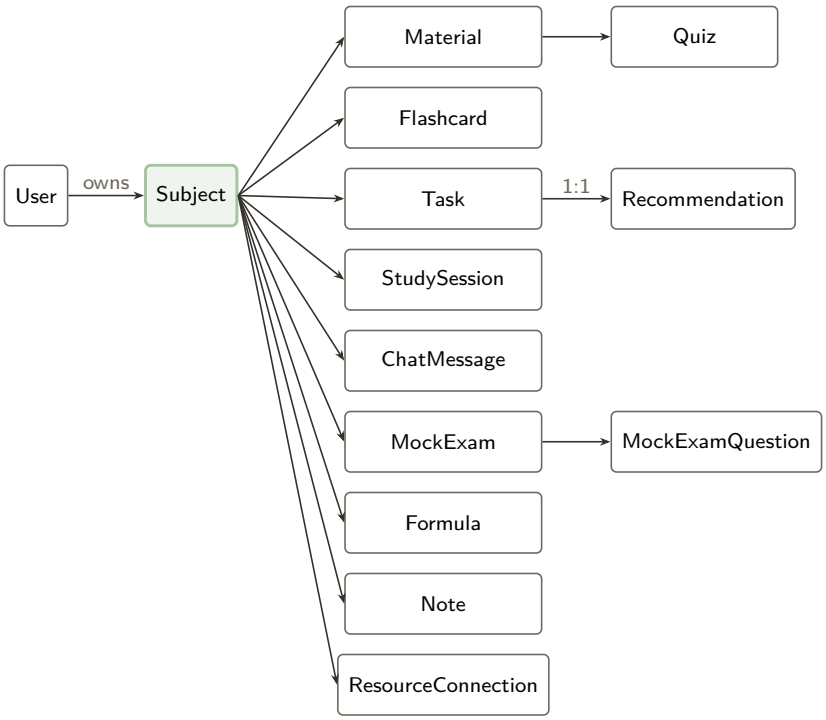


Figure 2: `User` owns `Subject`; almost everything else hangs off `Subject` with `ON DELETE CASCADE`.

Key fields worth knowing:

- **User** — `groq_api_key_encrypted` (Fernet-encrypted BYOK key, NULL = still on trial), `trial_requests_used` (counts free AI actions).
- **Subject** — `exam_date`, `priority_level` (1–5), `difficulty` (1–5), `confidence_score` (0–100). These feed the recommendation formula.
- **Material** — `summary`, `key_concepts` (JSON), `learning_complexity`, `importance_level`, `deep_research_summary`.

- **Flashcard** — `box` (Leitner 1–5) + `next_review_date` drive spaced repetition.
- **ResourceConnection** — `source_material_id` → `target_material_id` with a `connection_type` (Prerequisite / Extension / Foundational): the AI-generated knowledge-map edges.

4.1 Schema migrations

There is no Alembic. On startup `main.py` runs `Base.metadata.create_all()` to create missing tables, followed by a series of idempotent `ALTER TABLE ... ADD COLUMN` statements wrapped in try/except (e.g. adding `deep_research_summary`, `groq_api_key_encrypted`, `trial_requests_used`). This lightweight approach keeps the SQLite file forward-compatible across redeploys without a migration tool.

5 Authentication

`backend/app/auth.py`, intentionally dependency-free (hashlib / hmac / secrets only):

- **Passwords:** PBKDF2-HMAC-SHA256, 260,000 iterations.
- **Tokens:** `base64url(payload).hex(hmac_sha256(secret, payload))`, 30-day TTL. Stateless — no server session store.
- **Secret:** `SECRET_KEY` from env, else generated once and persisted to `.auth_secret` next to the DB so sessions survive restarts.
- **Frontend:** token + user cached in `localStorage` (`fb_token` / `fb_user`). `authFetch` in `lib/api.ts` attaches the bearer token and, on a 401, clears the session and redirects to `/login`.
- **Admin:** `ADMIN_EMAILS` env allowlist gates `/api/admin/*`. Single-owner project — a plain email allowlist, no roles system.

6 AI agent layer

`backend/app/services/agents.py`. Every agent calls `run_llm()`, which targets the Groq model named by `GROQ_MODEL` at `temperature=0.3`, optionally in JSON mode, and raises `AIServiceError` on any failure rather than fabricating content.

Why the model is configuration. It was hardcoded as `llama-3.3-70b-versatile` in four places until Groq retired that model and every AI feature began returning `404 model_not_found` at once. A single `GROQ_MODEL` setting means the next retirement is an environment variable, not a code change. Any replacement must support native tool calling, which the RAG tutor depends on; verify with a test call before switching.

Agent	Job
SummarizationAgent	Exhaustive cited summary + key concepts + complexity / importance ratings (JSON). Emits Mermaid/ASCII diagrams and `[Slide N]` citations.
DeepResearchAgent	Under-the-hood mechanics, exam pitfalls, real-world case studies — Markdown enrichment supplement.
QuizGenerationAgent	Flashcards + multiple-choice quizzes with mandatory source citations and per-question explanations; `generate_more_items` adds more while avoiding duplicates.
MockExamAgent	Generates 3 open-ended exam questions; grades typed answers 0–100 on accuracy / completeness / terminology with constructive feedback.
FormulaExtractorAgent	Extracts formulas as LaTeX + variables + derivation steps (JSON).
CurriculumMapperAgent	Computes typed conceptual edges between materials → the knowledge map.
PlanningRecommendationAgent	*Not an LLM call* — deterministic Python scoring of tasks (see below).

6.1 Recommendation score

`PlanningRecommendationAgent.calculate_recommendations` ranks pending tasks:

```
score = (exam_urgency  * 0.4) + (topic_importance * 0.3)
      + (low_confidence * 0.2) + (incompletion    * 0.1)
```

Urgency is derived from days remaining to `exam_date` ($\leq$1 day $\rightarrow$ 10, decaying with distance); low-confidence from `100 - confidence_score`. The result is scaled to 0–100 for display. The lowest-confidence topic with the nearest exam floats to the top.

7 Ingestion pipeline

`POST /api/materials/upload` (`main.py`). Upload is a 7-step pipeline; steps 1–2 run synchronously in the request, steps 3–7 run in a FastAPI `BackgroundTask` so the HTTP response returns immediately with a `job_id`.

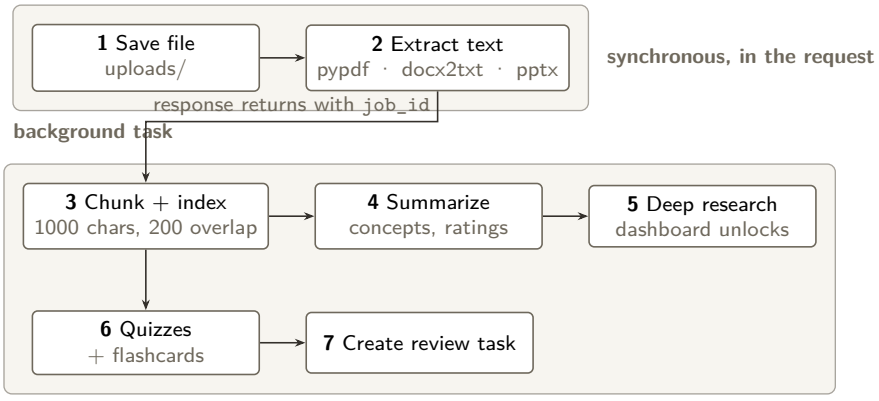


Figure 3: Steps 1–2 run inside the request so the client gets a job id immediately; the expensive AI work continues in the background and streams progress over SSE.

Progress streams to the client over **Server-Sent Events** at `GET /api/materials/upload-progress/{job_id}`. The free-trial action is consumed *only after* the background pipeline succeeds, so a failed digestion never burns a user's allowance.

Because the background thread runs after the response is sent, it cannot see the request's contextvar; the resolved Groq key is passed explicitly and re-bound with `set_request_key()` inside the thread (see §9).

8 RAG tutor

`backend/app/services/langchain_chat.py`, entered from `POST /api/tutor/chat`.

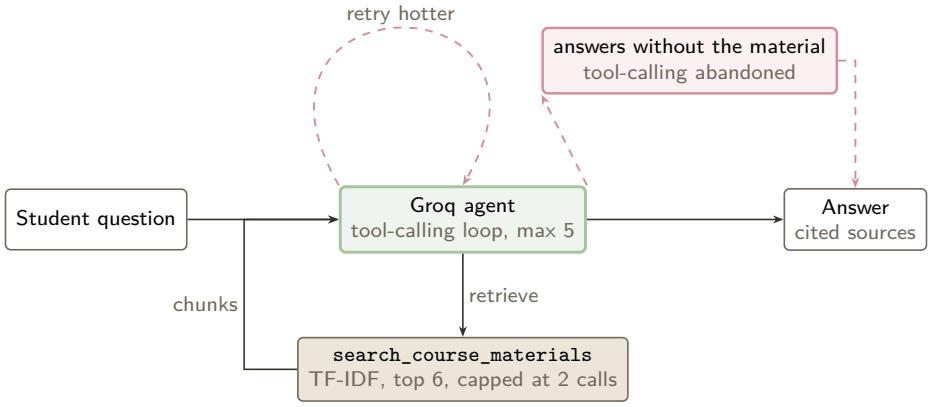


Figure 4: The two dashed paths still return a confident answer to the student. The right-hand one answers without consulting their material at all, which is why it is counted rather than merely logged.

- **Retrieval:** `vector_store.py` is a from-scratch **TF-IDF cosine** store persisted as `vector_store.json` — no external vector DB, no embedding API. It computes IDF over the filtered document set at query time and returns the top-k chunks. Documents are filtered by `subject_id` so a query only ever sees the current subject's material.
- **Agent loop:** a Groq tool-calling loop (max 5 iterations) with three tools — `search_course_materials`, `get_subject_info`, `get_study_progress`. Search is hard-capped at 2 calls per message.
- **Payload size:** each retrieved chunk is capped at 800 characters, which is what bounds the context sent to Groq. An ONNX text compressor (Headroom) used to sit here; it was removed after measurement showed it was compressing nothing (see §12).
- **Personas (`mode`):** `standard`, `simplified` ("explain like I'm 5"), and `analogies` inject different system-prompt suffixes.
- **Resilience:** on a malformed tool call the loop retries at escalating temperature (greedy decoding would just reproduce the same bad call); if tool-calling can't be recovered it falls back to a direct no-tools answer. In trial mode only, a 429 rate-limit spills over to a second server key (`GROQ_API_KEY_2`) — a personal key never spills onto the owner's account.

Every resilience path above is counted. Each one still returns a confident answer to the student, so none of them shows up as an error. A run that falls back to a no-tools answer has stopped using the student's material entirely while still looking healthy. The metrics in §12 exist to make those paths visible.

9 BYOK + free-trial model

`backend/app/key_context.py`. The app used to share one server Groq key for every visitor; this module makes AI usage freemium so hosting cost stays flat.

- New accounts get `FREE_TRIAL_LIMIT = 5` AI actions on the **server** key.
- After that, the user saves a personal Groq key (via `PUT /api/account/groq-key`), which is validated with a live `models.list()` call and stored **Fernet-encrypted** at rest. The Fernet key is derived (SHA-256) from the app's existing `SECRET_KEY`, so there is no new secret to manage. Keys are never returned in full — only a masked hint (`gsk_...4f9a`).
- **Key routing:** the resolved per-request key is stashed in a `contextvars.ContextVar`; `agents.run_llm` and the tutor read it via `get_current_key()`. This threads a per-user key through module-level Groq client singletons without rewriting every agent signature. `contextvars` are per-thread/task, so it is safe under FastAPI's sync threadpool and async workers.
- **Gate:** `ai_action()` (context manager) resolves the key, enforces the cap (HTTP 402 when exhausted, 503 when no key exists), and increments the trial counter *only* on success. `count=False` covers auto-fired paths like recommendations that should use a key but not consume the allowance.

9.1 Runtime key overrides

`config.py` supports a `.runtime_overrides.env` file: the admin dashboard's "rotate Groq key" feature writes here (in the persisted data volume) instead of editing the container's real `.env`, so key changes survive `docker rm` + redeploy without rebuilding the image.

10 HTTP API reference

All routes are under `/api`. All except signup/login require a bearer token. Ownership is enforced per subject (`own_subject` / `assert_owner`).

10.1 Auth & account

Method	Path	Purpose
POST	`/auth/signup`	Register, returns token
POST	`/auth/login`	Log in, returns token
GET	`/auth/me`	Current user
GET	`/account`	Account + trial/key status
PUT	`/account/groq-key`	Save/validate personal Groq key
DELETE	`/account/groq-key`	Remove personal key (back to trial)

10.2 Subjects

Method	Path	Purpose
POST	`/subjects`	Create subject
GET	`/subjects`	List subjects
GET	`/subjects/{id}`	Subject dashboard payload
PATCH	`/subjects/{id}`	Update (exam date, priority, confidence...)
DELETE	`/subjects/{id}`	Delete (cascades)

10.3 Materials & knowledge map

Method	Path	Purpose
POST	`/materials/upload`	Upload + start ingestion, returns `job_id`
GET	`/materials/upload-progress/{job_id}`	SSE progress stream
GET	`/subjects/{id}/materials`	List materials
PATCH/DELETE	`/materials/{id}`	Edit / delete material
POST	`/subjects/{id}/generate-map`	Build knowledge-map edges
GET	`/subjects/{id}/map`	Fetch nodes + edges

10.4 Study aids

Method	Path	Purpose
GET/POST	`/subjects/{id}/flashcards`	List / add flashcards
PUT/DELETE	`/flashcards/{id}`	Edit / delete
POST	`/flashcards/{id}/review`	Record review, advance Leitner box
GET/POST	`/subjects/{id}/quizzes`	List / add quizzes
PUT/DELETE	`/quizzes/{id}`	Edit / delete
POST	`/quizzes/{id}/answer`	Submit answer, get grading
POST	`/subjects/{id}/generate-more`	Generate more flashcards/quizzes
*	`/subjects/{id}/mock-exams`, `/mock-exams/{id}`	Mock exams (CRUD)
POST	`/mock-exams/{id}/submit`	Submit + auto-grade
GET/POST	`/subjects/{id}/formulas`, `/formulas/generate`	Formula sheets
*	`/formulas/{id}`	Edit / delete / annotate

10.5 Tasks, sessions, notes, tutor, dashboard

Method	Path	Purpose
*	`/tasks`, `/subjects/{id}/tasks`, `/tasks/{id}`	Planner tasks
*	`/study-sessions`, `/subjects/{id}/sessions`	Focus sessions
*	`/subjects/{id}/notes`, `/notes/{id}`	Rich notes
POST	`/tutor/chat`	RAG tutor (query + persona mode)
*	`/subjects/{id}/chats`, `/chats/{id}`	Tutor history
GET	`/dashboard/recommendations`	Ranked next actions
GET	`/dashboard/summary`	Aggregate stats

10.6 Admin (`ADMIN_EMAILS` only)

`GET /admin/overview`, `GET /admin/logs`, `GET /admin/health`, `GET|PUT /admin/config` — usage stats, app logs (from the stdout tee), health checks, and live Groq-key rotation.

11 Frontend

Next.js App Router. Three main routes:

- `/` **(page.tsx)** — auth gate + subject list/creation.
- `/subject/[id]` **(page.tsx, ~3800 lines)** — the study workspace: dashboard, materials, flashcards, quizzes, mock exams, formula sheets, knowledge map, notes, tutor chat, focus mode. This is where most of the product lives.
- `/admin` — admin dashboard (usage, logs, health, key rotation).

`lib/api.ts` is the single typed API client: session helpers, `authFetch` (bearer + 401 redirect), and `ApiError` (carries HTTP status so callers can branch, e.g. 403 → "not authorized"). `API_BASE` comes from `NEXT_PUBLIC_API_URL` (defaults to `http://127.0.0.1:8000/api`). Notes use the TipTap-based `NotionEditor` with math/KaTeX support.

12 Observability

`backend/app/observability.py` defines every metric; they are exposed at `GET /metrics` and scraped by Prometheus every 15 seconds. The existing code was touched in four places only, each a single line or a thin wrapper.

The point is to measure the *model*, not just the server. Uptime and CPU say nothing about whether the tutor is still using the student's material.

Metric	What it catches
`fb_llm_no_tools_fallback_total`	The agent answered without the student's materials. Still a 200 response.
`fb_llm_tool_validation_failures_total`	Groq rejected the model's tool call and it was retried hotter.
`fb_llm_rate_limit_switch_total`	The primary key was exhausted and traffic moved to the backup.
`fb_llm_search_limit_intercept_total`	The agent hit the hard two-search cap.
`fb_rag_top1_similarity`	Best match score per query. A falling p50 means questions are drifting off-corpus.
`fb_rag_zero_match_total`	Nothing indexed for that subject at all.
`fb_rag_empty_result_total`	Material exists but nothing overlapped the question. Deliberately separate from the above.
`fb_rag_payload_tokens`	Estimated context tokens per request.
`fb_llm_duration_seconds`	End-to-end agent time, versus `fb_rag_query_duration_seconds` for retrieval alone.
`fb_tutor_rolling_satisfaction`	Share of thumbs-up over the last 100 rated answers, fed by `POST /api/tutor/feedback`.

12.1 Two rules the instrumentation follows

Metrics can never break a request. Every recorder is wrapped in a `safe()` decorator that swallows its own exceptions, with a test that feeds deliberately malformed values into each one. A student must not get a 500 because a histogram bucket was misconfigured.

HTTP metrics label by route template, not path. `/api/subjects/41` and `/api/subjects/42` both report as `/api/subjects/{subject_id}`. Labelling by raw path mints a new time series per subject id, which is the standard way to bury a Prometheus instance.

12.2 What it found immediately

Within the first hour of real traffic the metrics surfaced three live bugs that logs, uptime and error rates had all missed:

1. **A retired model.** `llama-3.3-70b-versatile` returned `404 model_not_found`; every AI feature had been broken.
2. **A failover that had never worked.** `fb_llm_rate_limit_switch_total` equalled the error count exactly: the primary key hit its limit, failed over to a backup key, and the backup returned 401. The failover made things worse, turning a retryable 429 into a hard auth error.
3. **A dependency doing nothing.** The ONNX text compressor could never initialise, because its model needs PyTorch and torch was deliberately excluded from the image. Measured at 0 % saving across every model and ratio. Removing it took the image from 1.31 GB to 546 MB.

12.3 Dashboard

Grafana is provisioned from `infra/k8s/grafana-dashboard.json` with its datasource wired automatically, so the dashboard exists on a fresh cluster with no manual setup. Representative figures from the running system:

Measure	Value
Retrieval latency	1.2 ms
Full agent run	4.0 s average
Context per retrieval	≈1,000 estimated tokens
Retrieval gate	precision@1 0.875, recall@3 1.000

Retrieval is roughly four orders of magnitude faster than the model call, so all perceived latency is the Groq round trip. Optimising the retriever would be invisible to a student.

13 Testing

`backend/tests/`, run by CI on every push.

`test_rag_eval.py` is a retrieval quality gate rather than a unit test. Eight fixed questions each name the chunk that should rank first, scored against a small committed corpus. The build fails below precision@1 0.75 or recall@3 0.875. The corpus is synthetic on purpose: evaluating against the live vector store would make the expected answers move whenever a student uploads a file, and a test whose expected answer drifts is not a test.

It also guards three invariants: a student must never retrieve another subject's material, an off-topic question must not produce a confident match, and an empty corpus must return nothing rather than erroring.

`test_observability.py` covers metric recording, including the rule that a malformed value must never raise.

CI installs only numpy and prometheus-client for these, not the full requirements file, so the gate finishes in seconds instead of waiting on unrelated dependencies.

14 Configuration

Backend env (`backend/.env`, see `.env.example`):

Var	Purpose
`GROQ_API_KEY`	Server key for the free trial
`GROQ_API_KEY_2`	Rate-limit fallback (trial mode only)
`GROQ_MODEL`	Groq model id, default `qwen/qwen3.8-27b`. Must support native tool calling
`DATABASE_URL`	Defaults to `sqlite:///./finals_buddy.db`
`SECRET_KEY`	Token signing + BYOK encryption seed (auto-generated if unset)
`CORS_ORIGINS`	Comma-separated allowed frontend origins
`ADMIN_EMAILS`	Comma-separated admin allowlist
`RUNTIME_OVERRIDES_PATH`	Path to the runtime key-override file
`PORT / HOST`	Server bind (default `0.0.0.0:8000`)

Frontend env: `NEXT_PUBLIC_API_URL` → backend `/api` base.

15 Running locally

Backend:

```
cd backend
python -m venv .venv && source .venv/bin/activate   # Windows: .venv\Scripts\
    activate
pip install -r requirements.txt
cp .env.example .env        # add GROQ_API_KEY
python -m uvicorn app.main:app --reload --port 8000
```

API docs (Swagger) at `http://127.0.0.1:8000/docs`.

Frontend:

```
cd frontend
npm install
# .env.local: NEXT_PUBLIC_API_URL=http://127.0.0.1:8000/api
npm run dev        # http://localhost:3000
```

16 Deployment

- **Backend:** Docker image (`backend/Dockerfile`), `python:3.12-slim`, architecture-agnostic across x86_64 and ARM64. The app *runs from* `/srv/finals-buddy/data` (its `WORKDIR`) while code lives at `/srv/finals-buddy`, so every relative artifact — `finals_buddy.db`, `uploads/`, `vector_store.json`, `.auth_secret`, `.runtime_overrides.env`, `logs/app.log` — lands in one mountable volume:

  ```
  docker run -v finals_data:/srv/finals-buddy/data --env-file .env \
             -p 8000:8000 finals-buddy
  ```

  Uvicorn runs with `-proxy-headers -forwarded-allow-ips=*` behind a reverse proxy.

- **Frontend:** Vercel; `NEXT_PUBLIC_API_URL` points at the deployed backend.

- **Logs:** stdout/stderr are teed to `logs/app.log` (size-capped) so the admin dashboard can show `docker logs`-equivalent output without mounting the Docker socket into the container.

16.1 Infrastructure as code

`infra/terraform/` describes the whole environment: an EC2 instance running k3s, an ECR repository, IAM roles, and SSM parameters for secrets. The notable choices:

- **k3s on one EC2 instance, not EKS.** The EKS control plane costs $0.10/hour regardless of load. A single-node k3s cluster gives the same Kubernetes API and the same manifests for the price of the instance. This is not high availability and does not pretend to be; what it buys is declared state, health probes, rolling image updates and one-command rollback.

- **No SSH key, port 22 closed.** Deploys and any manual access go through AWS Systems Manager, authenticated by IAM. There is no private key to store in GitHub.

- **Secrets in SSM Parameter Store, encrypted.** Terraform creates each parameter with a placeholder and then ignores its value permanently via `lifecycle { ignore_changes = [value] }`. Real values are written out of band by `infra/scripts/populate-secrets.sh`, which pipes them straight into the AWS CLI without echoing them. No key reaches git or Terraform state, and rotating one no longer requires editing a file on a server.

- **Local Terraform state.** Remote state with locking solves concurrent applies by multiple operators. There is one operator.

A `systemd` timer on the instance refreshes the ECR pull token (which expires every 12 hours) and re-syncs secrets from SSM into a Kubernetes secret.

16.2 Kubernetes

`infra/k8s/` holds four workloads: the API, Caddy, Prometheus and Grafana. Two decisions are specific to this app:

- **A persistent volume, because the app is stateful.** The backend writes `finals_buddy.db`, `uploads/` and `vector_store.json` into its working directory. Without the volume, every pod restart would wipe a student's materials and the entire retrieval corpus.
- **Strategy is `Recreate`, not `RollingUpdate`.** SQLite on a `ReadWriteOnce` volume tolerates exactly one writer, and a rolling update briefly runs two pods. That trades a few seconds of downtime for not corrupting the database. It is a consequence of SQLite; moving to Postgres would remove it.

Caddy terminates TLS for the public hostname and obtains Let's Encrypt certificates automatically, with its certificate store on its own volume so a restart does not re-request them and trip rate limits. A plain-HTTP catch-all block lets the pipeline smoke-test the instance by IP before DNS points at it.

16.3 CI/CD

`.github/workflows/ci-cd.yml`. A push to `master` is the only path to production; `workflow_dispatch` can deploy from another branch to prove the pipeline without merging.

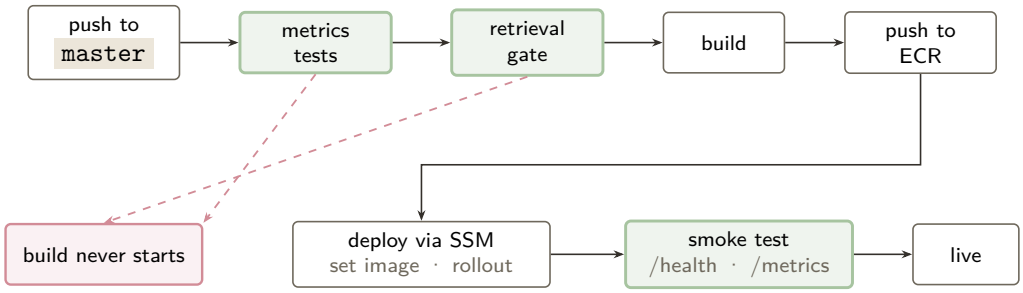


Figure 5: Two blocking gates sit before anything is built. A retrieval regression fails the run while the only artefact is a test report.

GitHub authenticates to AWS through OIDC: each run receives a short-lived token, and the trust policy accepts it only from this repository. No AWS credential is stored in GitHub at all. The deploy role can push to one ECR repository and send one SSM document to one instance, and nothing else.

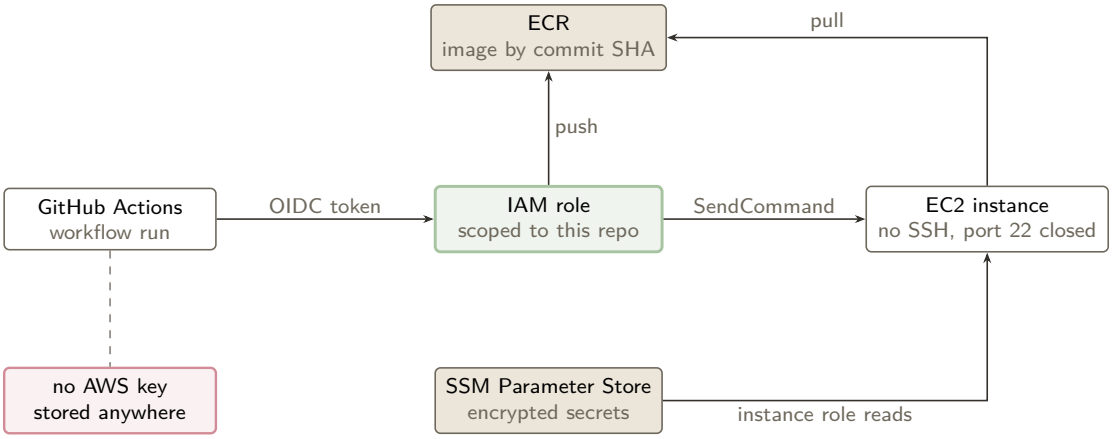


Figure 6: The only thing GitHub holds is a role ARN. Secrets live encrypted in SSM and are read by the instance itself, never passed through the pipeline.

An IAM subtlety worth knowing. `ssm:SendCommand` supports resource-level permissions and is scoped to one instance and one document. `ssm:GetCommandInvocation` does *not* support them, so scoping it the same way denies it outright and the pipeline fails after a successful deploy. It has to be granted on `*`. Least privilege means knowing which actions can be scoped, not narrowing every one of them reflexively.

The smoke test checks `/metrics` as well as `/health`, because a deploy that silently drops instrumentation leaves every dashboard flat, and flat reads as quiet rather than broken.

The deployed image is tagged with the git commit SHA, so what is running traces back to an exact commit with no manual build in the chain.

16.4 Migration history

The backend previously ran as a single Docker container on an Oracle Cloud always-free VM, deployed by a GitHub Action holding a long-lived SSH key. That box was a 1 GB shape with under 500 MiB free and already swapping, which could not host k3s plus a monitoring stack without killing the application.

The move to AWS was performed as a DNS cutover: infrastructure, secrets, pipeline, verification and seeding all happened while the old box still served every user, and the only user-visible step was repointing one A record. The old environment was left running throughout, so rollback was the same single record.

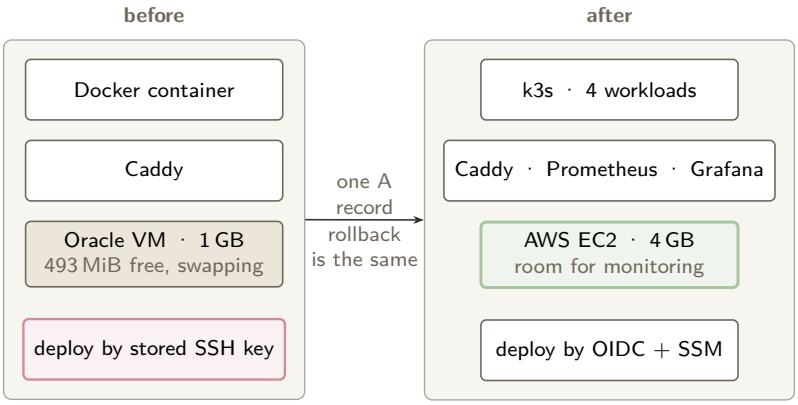


Figure 7: The migration's only user-visible step, and its rollback, are the same single DNS change.

For the original Oracle runbook see `DEPLOY.md` (not tracked in the public repo); it is superseded by `infra/` but documents the manual path.